

Project — Multi-provider chatbot

LESSON 06 · MODULE 1 CAPSTONE — FOUNDATIONS

What you'll learn

- How to compose two agents into a small system: a classifier that routes, a responder that answers
- Why splitting routing and answering into two agents beats one agent doing both
- How structured outputs (video 05), provider swapping (video 03), and streaming (video 04) compose into a real pipeline
- The "plan, then fan out" architectural pattern you'll reuse across every project from here on

Key concepts

Two agents, one chat loop. First agent is the *classifier* — structured output. It looks at the message and returns a typed routing decision: what kind of request is this, which provider should handle it. Second agent is the *responder* — provider swapping + streaming. It gets instantiated fresh with whichever provider the classifier picked, and streams the answer back.

Why two agents instead of one? Routing is a different job than answering. Classification needs a predictable typed output — perfect fit for `response_model`. Answering needs a conversational stream — perfect fit for `stream=True`. One agent trying to do both would be worse at both.

Reach for small specialized agents. It's a pattern you'll come back to. Each agent has one job. You can swap the classifier for a smarter version without touching the responder. You can add new providers to the routing table without changing the loop. You can repoint the classifier at a different decision schema — say, routing between tools instead of providers — and the same architecture holds.

Responder is a function, not a persistent agent. The provider changes per turn, so we spin up a fresh agent every call. `create_agent` is just a config call — cheap, stateless.

The hard-coded routing table is the demo, not the design. You tune it for your own use case. The point is that one decision (classifier) is type-shaped and reproducible, and the other (responder) is free-form and creative — and the architecture separates them cleanly.

TAKEAWAY

Two agents, one chat loop.

TAKEAWAY

Why two agents instead of one?.

TAKEAWAY

Reach for small specialized agents.

TAKEAWAY

Responder is a function, not a persistent agent.

TAKEAWAY

The hard-coded routing table is the demo, not the design.

Code shown in this lesson

PYTHON

```
from pydantic import BaseModel, Field
from typing import Literal

# Classifier – structured output (video 05)
class RoutingDecision(BaseModel):
    intent: Literal["creative", "factual", "urgent", "speed"]
    provider: Literal["claude-sonnet", "gpt-4o", "gpt-4o-mini", "grok-llama-3.3-70b"]
    reason: str = Field(description="One sentence.")

classifier = studio.create_agent(
    name="router",
    provider="gpt-4o-mini", # fast and cheap for routing
    role="request classifier",
    goal="pick the right model for each user message",
    instructions=(
        "Creative → claude-sonnet. Factual → gpt-4o. "
        "Urgent → claude-sonnet (apologetic). Speed → grok."
    ),
    response_model=RoutingDecision,
)
```

PYTHON

```
# Responder – fresh agent per turn, with the picked provider, streamed
def stream_answer(provider: str, message: str):
    responder = studio.create_agent(
        name="responder",
        provider=provider,
        role="helpful assistant",
        goal="answer the user clearly",
        instructions="Be direct. No preamble.",
    )
    for chunk in responder.run(message, stream=True):
        if chunk.done:
            break
        print(chunk.content, end="", flush=True)
```

PYTHON

```
# Main loop – classify, route, stream
def chat(message: str):
    decision = classifier.run(message) # typed RoutingDecision
    print(f"→ routing to {decision.provider} ({decision.reason})")
    stream_answer(decision.provider, message)
```

TRY THIS

- 1 Add a fifth intent — `"analytical"` — to the `RoutingDecision` model, mapped to `gpt-4o-mini` in the classifier's instructions. Feed it three analytical questions ("Summarize this CSV", "What's the trend?") and prove the classifier picks the right provider.

2

Replace the hard-coded routing table with a per-user preference dict (`{"creative": "claude-sonnet", "factual": "gpt-4o", "default": "gpt-4o"}`). Make the classifier honor those preferences in its instructions. Two users, same prompt, different providers — the architecture should hold.